`code/` Toolbox — Reference Manual

MATLAB accelerator-physics toolbox for linac / ERL / arc design: lattice I/O, first- and second-order transport, Twiss & space-charge envelopes, longitudinal compression, CSR / wakefield modeling, microbunching-instability (MBI) gain, BBU tracking, and beam-file analysis.

Shared object — the lattice. Almost everything operates on a *beamline*: a struct array of elements built by `createElement` (or parsed by `readBmad`). The element struct and the per-element 6×6 matrices come from `createElement.m` / `getElementMatrix.m` (§1);
read those first — they are the contract every other file relies on.

Conventions: lengths [m], energies in MeV or eV (per-function, noted below), angles [rad] unless a name ends in `_deg`, charge [C]. Phase-space vector is `[x; px; y; py; z; delta]`.

Contents

---

1. Element model & core transport

The core functions to build a lattice, get its transport matrices, and propagate Twiss.

FILE	SIGNATURE	PURPOSE
`createElement.m`	`el = createElement(type, length, varargin)`	Build one element struct with defaults; `type` ∈ `drift`, `dipole`, `quadrupole/quad`, `sextupole/sext`, `edge`, `marker`, `cavity`, `tgu`, `1stttay`. Trailing `varargin` sets type-specific params (dipole `h`, quad `k1`, edge `h,angle`, cavity gradient/freq/phase, …) and an optional name.
`calculateTotalMatrix.m`	`[R, gamma_out] = calculateTotalMatrix(beamline, gamma_in)`	Cumulative 6×6 first-order transport matrix of a whole beamline. Pass `gamma_in` to track energy through cavities (`gamma_out` = exit value).
`nonlinear_calculator.m`	`Tijk = nonlinear_calculator(bl, i,j,k[,delta,E0])` / `[s,T]=...(bl, ilist,jlist,klist,...)`	Second-order Taylor map terms `Tijk` (e.g. `T566`) by finite-difference particle tracking. Single-point mode (1 output) or along-the-line evolution mode (2 outputs). `E0` in eV, default `4e6`.

FILE	SIGNATURE	PURPOSE
`calcBetaEvolution.m`	`[final_beta, evolution, periodic_twiss] = calcBetaEvolution(bl, E_MeV, init_beta, plot_flag, calc_periodic)`	Twiss (β,α) evolution through a line; optional periodic-cell solution and R16 tracking. `init_beta=[βx,βy,αx,αy]`; `final_beta` same layout; `evolution` carries `s`/β/α/R-element arrays.

Element struct fields (set by `createElement`, consumed by `getElementMatrix`): `type`, `length`; bends use `h = 1/rho` (or `angle`); quads use `k1`; sextupoles `k2`; edges `h, angle`; cavities `gradient`/`voltage`, `phase` [deg], `frequency`, `num_cells`.

Example — build a beamline and compute its matrices

```
% 1) Build the beamline element-by-element with createElement
d1  = createElement('drift', 0.5);                % 0.5 m drift
q1  = createElement('quadrupole', 0.2, 1.5);      % L=0.2 m, k1=+1.5 m^-2
rho = 5;  ang = 0.10;                              % bend radius 5 m, angle 0.1 rad
b1  = createElement('dipole', ang*rho, 1/rho, 'B1');   % length = ang*rho, h = 1/rho
q2  = createElement('quadrupole', 0.2, -1.5);     % defocusing quad

beamline = [d1, q1, b1, q2, d1];                  % a struct array = the lattice

% 2) First-order 6x6 transport matrix of the whole line
R    = calculateTotalMatrix(beamline);
R56  = R(5,6)                                      % linear momentum compaction [m]

% 3) Second-order longitudinal term T566 (delta step 1e-3, E0 = 1 GeV in eV)
T566 = nonlinear_calculator(beamline, 5,6,6, 1e-3, 1000e6)
```

`R` is the 6×6 matrix in `[x; px; y; py; z; delta]`; pick any element, e.g. `R(1,2)` (`x`→`x'` term) or `R(5,6)` (`R56`). For a lattice with cavities, call `calculateTotalMatrix(beamline, gamma_in)` so the energy is tracked.

Example — Twiss (β-function) evolution

```
% Propagate Twiss through the same beamline at 1 GeV, with a plot.
init_beta = [10, 10, 0, 0];                % [betax betay alphax alphay] at entrance
[final_beta, evo] = calcBetaEvolution(beamline, 1000, init_beta, 1, 0);   % plot_flag=1

betax_max = max(evo.beta_x);                % e.g. inspect the evolution arrays
fprintf('exit betax = %.3f m,  betay = %.3f m\n', final_beta(1), final_beta(2));

% For a periodic cell, ask for the matched (periodic) solution instead:
[~, ~, periodic_twiss] = calcBetaEvolution(beamline, 1000, init_beta, 1, 1);  % calc_periodic=1
```

---

2. Lattice I/O

The three ways to move a lattice between disk, the `beamline` struct array, and editable `createElement` source.

FILE	SIGNATURE / KIND	PURPOSE
`readBmad.m`	`beamline = readBmad(bmadFile, verbose)`	Parse a Bmad `.bmad`/`.arc` lattice (handles `&` continuations, `use`, line expansion, dipole edges) into a beamline struct array. Emits types `drift, quadrupole, dipole, sextupole, edge, marker, cavity, multipole`.

FILE	SIGNATURE / KIND	PURPOSE
`writeBmad.m`	`bmadCode = writeBmad(beamline)`	Inverse of `readBmad`: turn a beamline back into Bmad text (auto-suffixes repeated element names). Returns the string and prints it to the console.
`BeamlineToEle.m`	`BeamlineToEle(beamline)` (prints)	Turn a beamline into editable `createElement` **source** — one line per element (all types, with names), plus the `beamline = [...]` assembly line. Copy the output into a script and run it to rebuild/tweak the lattice.

Example — read a lattice, regenerate editable source, write it back

```
% 1) Parse a Bmad lattice into a beamline struct array
beamline = readBmad('CSRwake/example/pap4.arc');   % set verbose=true for a parse summary

% 2) Print createElement source (paste into a script to edit by hand)
BeamlineToEle(beamline);
%   -> D1 = createElement('drift', 0.5, 0, 'DR01');
%      Q1 = createElement('quadrupole', 0.2, 1.5, 'QF1');
%      B1 = createElement('dipole', 0.43, 0.1, 'B1');
%      ...
%      beamline = [D1, Q1, B1, ...];

% 3) Convert the (possibly edited) beamline back to Bmad text
bmadText = writeBmad(beamline);              % also echoes to the console
```

> ***Round-trip caveat:*** *`readBmad`'s internal standardization keeps only `name,type,length,k1,k2,angle,h,e1,e2,voltage,frequency,phase`, so a* ***cavity's*** *`gradient/num_cells` **are not recovered** from a `.bmad` file — `BeamlineToEle` flags this with a `NOTE` so you can fill them in before rebuilding.*

---

3. Space-charge (SC) envelope & slice analysis

Slice space-charge envelope integrator (ODE with SC defocusing), plus the reader that builds the slice-parameter file it consumes.

FILE	SIGNATURE	PURPOSE
`read_z_distribution.m`	`slice_data = read_z_distribution(infile, num_slices, threshold_n, n_sigma)`	Read a Bmad beam file, apply an n-σ transverse cut, slice in `z`, and return the per-slice Twiss/emittance/charge matrix. Row 1 holds globals (`col16=Q`, `col17=E0`); rows 2..end-1 are slices. This matrix (saved to a text file) is the input `TSCenv` expects.
`TSCenv.m`	`results = TSCenv(bl, E0, beamfile1)`	Slice SC envelope that additionally includes **velocity (drift) bunch compression** in `Ksc` (`R56=-L/(β²γ²)`); binds same-named quad gradients globally. `E0` in eV; reads `Q` from the slice file `beamfile1`. Returns a `results` struct with fields `s_array`, `sigx_array`, `sigy_array`, `sigpx_array`, `sigpy_array`, `gamma_array`, `emitx_proj_array`/`emity_proj_array`, `k1_values`, `peak_slice_emitx`/`y`, and centroids.

Benchmark — vs. RF-Track / Ocelot / ASTRA (the workflow in `SCmatch.m`)

`SCmatch.m` builds a match section + multi-cavity linac with `createElement`, runs `TSCenv`,
and overlays the envelope and normalized emittance against external tracking codes. The benchmark files `TT_RF_Track.txt`, `TT_Ocelot.txt`, `TT_Astra_TraceSpace.txt` use columns
`[s, _, _, εnx, εny, σx, σy]` (1,4,5,6,7).

```
% ... after building `beamline` as in SCmatch.m ...
res  = TSCenv(beamline, E_target, 'scmatch_slice_params.txt');
% E_target in eV
ref  = load('TT_RF_Track.txt');

% Beam-size benchmark: envelope (lines) vs RF-Track (points)
plot(res.s_array, res.sigx_array*1e3, 'r-', ...
```

```
     res.s_array, res.sigy_array*1e3, 'b-'); hold on
scatter(ref(:,1), ref(:,6), 25, 'r', 'filled');     % sigma_x [mm]
scatter(ref(:,1), ref(:,7), 25, 'b', 'filled');     % sigma_y [mm]
xlabel('s [m]'); ylabel('\sigma [mm]'); legend('env x','env y','RF-Track');

% Normalized-emittance benchmark (gamma * geometric proj. emittance, in mm·mrad)
figure;
plot(res.s_array, res.gamma_array.*res.emitx_proj_array*1e6, 'r-', ...
     res.s_array, res.gamma_array.*res.emity_proj_array*1e6, 'b-'); hold on
scatter(ref(:,1), ref(:,4), 25, 'r', 'filled');     % eps_nx
scatter(ref(:,1), ref(:,5), 25, 'b', 'filled');     % eps_ny
xlabel('s [m]'); ylabel('\epsilon_n [mm\cdotmrad]');
```

Run `SCmatch.m` directly for the full match+linac example with all three tracking codes overlaid.

---

4. CSR kick & chicane

The standalone CSR line-integral kernel and the chicane builder that uses it. (The multi-particle / slice CSR pipelines that wrap this kernel are in §5.)

`CalcCSRkick.m` — CSR kick line integrals

```
[bend_integrals, segment_integrals, min_R16] = ...
        CalcCSRkick(beamline, h_value, sigz, calculate_min_R16)
```

Raw ss-CSR bend integrals and inter-bend (segment) integrals, returned as `R51`/`R52`-weighted line sums. Reference kernel that `csr_calc_mp`/`csr_calc_slice` (§5) re-implement inline.

ARG	MEANING
`beamline`	element struct array (forward order; reversed internally)
`h_value`	compression factor in the kernel `C = 1/(1 − h_value·R56)`
`sigz`	bunch length used in the drift-segment `phi_0` formula [m]
`calculate_min_R16`	`0/1`; if `1`, also sample `R16 = R(1,6)` along the line and return its minimum

OUT	MEANING
`bend_integrals`	`[intB_R51, intB_R52]` — bend (steady-state) line sums
`segment_integrals`	`[sum_segment_R51, sum_segment_R52]` — inter-bend (drift) log sums
`min_R16`	minimum sampled `R16` (only if `calculate_min_R16==1`, else `0`)

Internals: `extractBendInfo` finds bends from `el.h`/`el.angle`/`el.type` (`rho=1/h`); `integrateBendElement_Fast` substeps each dipole (`n_steps=500`) with cached half/full-step
matrices; `calculateSegmentContribution` is the drift-CSR log term. The top function body
is exploratory scratch — **the value actually returned comes from** `CalcCSRkick_Fixed_Strict` (the clean, order-correct version); read that one when in doubt.

`schicanecalcu.m` — 4-dipole chicane + its CSR kick

```
[intB, R56, beamline, l3] = schicanecalcu(r, L1,L2,L3, D1,D2, h, is_round)
```

Build a 4-dipole chicane (bend radius `r`, segment/drift geometry), returning its `R56`, the assembled `beamline`, the solved-for drift `l3`, and its CSR bend integrals `intB` (computed via `CalcCSRkick` above). See the worked Example F (§11).

---

5. CSRwake toolkit

The `CSRwake/` code: CSR energy/length kicks on a bunch through a bending lattice (arc or chicane) plus longitudinal transport. Two pipelines share one physics model, differing only
in how the bunch is represented; both re-implement the `CalcCSRkick` kernel (§4) inline.

5.1 Overview

- `CSRwake/mp/` — **multi-particle** pipeline (`csr_calc_mp`, `csr_drift_mp`, drivers) (§5.2).
- `CSRwake/slice/` — **slice** (current-profile) pipeline (`csr_calc_slice`, drivers) (§5.3).

Both walk the lattice, accumulate the 6×6 matrix `R`, use `R56(s) = R(5,6)` as the local compaction, and add steady-state (in-dipole) plus drift/transient (inter-bend) CSR kicks — the same model as the `CalcCSRkick` kernel (§4).

> ***Reversed-walk convention.*** *All routines reverse the beamline before integrating (`rev_beamline`), so `R_cum(5,6)` at any point is the residual `R56` from that point to the end of the original line — the quantity that actually compresses the slice radiating there. `CalcCSRkick.m` defines this convention; the CSRwake routines follow it.*

5.2 `CSRwake/mp/` — multi-particle pipeline

Operates on raw particle arrays `z`, `delta`. Output current is built by **histogramming** transported particles (robust through caustics/folds).

`csr_calc_mp.m` — core CSR + transport

```
out = csr_calc_mp(z, delta, Q_total, beamline, gamma_rel, T566,
NAME,VALUE,...)
```

REQUIRED	MEANING
`z, delta`	`[N×1]` particle arrays
`Q_total`	total bunch charge [C]

REQUIRED	MEANING
`beamline`	lattice struct array
`gamma_rel`	Lorentz factor
`T566`	2nd-order longitudinal dispersion [m]; pass `[]` to auto-compute via `nonlinear_calculator`

OPTION	DEFAULT	MEANING
`sige`	`std(delta)`	energy spread used in kernel
`sigzf`	auto	final bunch length; auto = `std(z + r56·delta + T566·delta²)`
`n_substeps`	500	R56(s) samples per dipole
`drift_csr`	true	include dr-CSR (false → ss-only)
`n_bends`	0	if >0, only integrate the first N bends
`dR56`	0	added to R56 in the **transport step only** (not the integrals)
`n_grid`	200	grid points for `lambda(z)`
`smooth_win`	5	Gaussian smoothing window for the binned current
`interp`	'linear'	how the grid kick is interpolated back onto particles
`plot_flag`	1	draw the 4-panel diagnostic figure

Output: `table` with columns `z_out_noCSR, z_out_wCSR, delta_noCSR, delta_wCSR`, plus `out.Properties.UserData` (resolved scalars `sige_used`, `R56_file`, `r56_tot`, `sigma_z0`, `sigma_zf`, `S_R56`, `S_one`, `S_R56_dr`, `S_one_dr`, `k_csrD`, `n_bends_used`, and grids `z_grid`, `I_grid`, `lambda_grid`, `Gamma_grid`).

OTHER MP FILE	PURPOSE
`csr_drift_mp.m`	Same signature (minus `n_substeps/drift_csr`); `*_wCSR` columns hold **only** the drift-CSR contribution. Per-particle `dz_csr`, `ddelta_csr` in `UserData`.
`cavity_long.m`	`[z_out,delta_out,E_ref_out]=cavity_long(z_in,delta_in,E_ref_in,V,phi_deg,f)` — thin-cavity full-cosine RF kick + adiabatic damping; `z` unchanged; `phi_deg=0` on-crest.
`bmad2mp.m`	`[z,delta,Q_total,np]=bmad2mp(infile,source_file)` — `source_file=1`→BMAD ASCII; `=charge[C]`→ASTRA. Mean-centered `z`, `delta=(p−<p>)/<p>`.

OTHER MP FILE	PURPOSE

5.3 `CSRwake/slice/` — slice (current-profile) pipeline

Operates on slice arrays `z_slice, delta_slice, I_slice`. `lambda(z)` is built directly from slices (no histogram); kicks evaluated at slice centers. Output current uses the slice **Jacobian** `I0/|dz_out/dz0|` (faster, but noisy at caustics — hence the mp version).

SLICE FILE	PURPOSE
`csr_calc_slice.m`	`out = csr_calc_slice(z_slice,delta_slice,I_slice,beamline,gamma_rel,T566,NAME,VALUE,...)` — same physics/options as `csr_calc_mp` (no `n_grid`/`interp`); **current-weighted** moments. Columns `z0, z_out_noCSR, z_out_wCSR, delta_noCSR, delta_wCSR, dIdz_smooth` + same `UserData`.
`cavity_long.m`	Slice version of the RF cavity, plus optional 7th arg `series_order`: Taylor-truncate `cos(phi-kz)-cos(phi)` to that order (`0`=full cosine default; `2`=chirp+curvature).
`bmad2slice.m`	`[zc,mean_delta,I_slice,mean_delta_raw]=bmad2slice(infile,nslices,fit_order,min_particles,source_file)` — bins to `nslices`, **weighted** polynomial fit of `delta(z)` (order `fit_order`, weights = slice count), trims sparse edges (`min_particles`). `source_file` as in `bmad2mp`.
`run_csr_slice.m`	driver: `readBmad` → `bmad2slice` → `csr_calc_slice`, prints resolved scalars.
`run_acc_arc.m`	driver: cavity → arc, ss+dr vs ss-only on slices, BMAD-benchmark scaffolding (commented).

5.4 Typical CSRwake workflows

```
% --- multi-particle, from an ASTRA dump ---
addpath('..','..\..');                          % CSRwake/ and
code/ helpers
[z,delta,Q] = bmad2mp('Bunch.ast', 70e-12);  % charge in C
beamline    = readBmad('..\example\pap4.arc');
out = csr_calc_mp(z, delta, Q, beamline, 1000/0.511, [], ...
                  'n_substeps',200, 'drift_csr',true,
'plot_flag',1);
ud  = out.Properties.UserData;                % resolved scalars
+ grids

% --- slice, explicit T566 and transport-only dR56 ---
[zc,d,I] = bmad2slice('beam.ast', 300, 9, 10, 70e-12);
beamline = readBmad('..\example\pap4.arc');
out = csr_calc_slice(zc, d, I, beamline, E1/0.510998950, -1.37,
...
                     'smooth_win',30, 'dR56',-8e-4,
'drift_csr',true);
```

Or just edit a driver (`run_csr_mp.m` / `run_acc_arc.m`): change the `USER INPUTS` block and run. The mp/slice drivers `addpath('..')` and `addpath('..\..')` to reach the `code/` helpers — keep the `code/CSRwake/{mp,slice}` layout or fix the paths.

5.5 CSRwake conventions, gotchas & sign rules

- **Units:** `z` [m], `delta` dimensionless, charge [C], energies [MeV], frequencies [Hz]. Plots scale `z`→μm, `delta`→10^{-3}.
- **Bend detection** (`local_bend_info`/`extractBendInfo`) accepts `abs(h)>1e-10`, a nonzero
  `angle`, or a `type` ∈ `{dipole, sbend, rbend, bend, sector, rectangular}`.
- `sigzf` **auto vs override:** an explicit `sigzf` is a *floor* (`max(opt.sigzf, sigzf_auto)`),
  not an absolute override.
- `dR56` **only moves the transport step**, never the CSR line integrals; it does rescale
  `sige` proportionally when `R56_file ≠ 0`.

- **Past full compression:** if no-CSR transport gives `sigma_zf ≤ 0` the routines error —
  check the signs of `sige`, `R56`, `dR56`, `T566`.
- **Kernel blow-up:** when `1 – (sige/sigzf)·R56(s) ≤ 0` the kernel returns `NaN`; inspect
  `n_bends_used`/`UserData` if results are NaN.
- **mp vs slice output current:** mp = histogram (robust through folds); slice = Jacobian
  `I0/|dz'|` (cheap, noisy near caustics).
- `csr_calc_mp` **ignores extra R56-file name-value pairs** (`s_col`, `R56_col`) left over
  from an older R56-line-file interface; the lattice-walk path is authoritative now.

5.6 Worked example — BMAD benchmark (`CSRwake/example/`)

`CSRwake/example/PLOTbenchbottom.m` is a complete, runnable slice-CSR benchmark. It accelerates a BMAD bunch through a thin cavity, transports it through the `pap4.arc` arc with the slice pipeline (ss-only **and** ss+dr), and overlays the end-of-arc phase space and current against BMAD particle dumps (no-CSR / ss-CSR / all-CSR). It **calls** the slice functions in `CSRwake/slice/` (via `addpath`) and the `code/` helpers — nothing is duplicated. All data files live next to the script:

FILE (IN `CSRWAKE/EXAMPLE/`)	ROLE
`PLOTbenchbottom.m`	the benchmark script
`pap4.arc`	arc lattice (`readBmad`)
`pap4.beam`	input bunch (BMAD ASCII, `bmad2slice`)
`pap4outnc.dat`	BMAD reference — no CSR
`pap4outwc.dat`	BMAD reference — ss-CSR only
`pap4outwc2.dat`	BMAD reference — ss + dr (all) CSR

Run it from its own folder (so the relative `addpath('..\slice')`, `addpath('..\..')` and the local data-file names resolve):

```
cd CSRwake/example
PLOTbenchbottom        % -> 2-panel figure: end-of-arc phase space + current
```

Skeleton of what it does (edit the `USER INPUTS` block to retarget):

```
addpath('..\slice');  addpath('..\..');                          % call slice + code/ helpers
[z0,delta0,I0] = bmad2slice('pap4.beam', 300, 9, 10, 1);     % §5.3  slice the dump
[z1,delta1,E1] = cavity_long(z0, delta0, 1000, V_cav, 13.55, 1.3e9);  % §5.3  cavity
beamline  = readBmad('pap4.arc');                                % §2    arc lattice
gamma_rel = E1 / 0.510998950;
out_total = csr_calc_slice(z1,delta1,I0, beamline, gamma_rel, -1.3, 'drift_csr',true);  % ss+dr
out_ss    = csr_calc_slice(z1,delta1,I0, beamline, gamma_rel, -1.3, 'drift_csr',false); % ss
% then read pap4out*.dat and overlay phase space + current (slice Jacobian I0/|dz'|).
```

---

6. Microbunching instability (MBI,test)

FILE	SIGNATURE	PURPOSE
`MBI_calc.m`	`result = MBI_calc(beamline, beam, opts)`	Self-contained Volterra-integral MBI gain solver (Tsai, PRAB 19, 114401). `beam` struct = energy/current/emittance/Twiss/σδ/σz/chirp; `opts` toggles CSR ss/tr/drift, shielding, transverse LD, λ scan. Returns gain spectra `Gf_dd/de/ed/total` and map `G_vs_s`.
`MBI_example.m`	script	Worked example: builds a chicane with `createElement`, sets `beam`/`opts`, runs `MBI_calc`, prints peak gain. Start here.

FILE	SIGNATURE	PURPOSE
`MBI_cal/`	folder	The same model split into one-function-per-file (`compute_Z_csr`, `csr_ss`, `csr_tr`, `csr_drift`, `csr_shielding`, `lsc_impedance`, `F0_shielding`, `local_s_in_dipole`, plus its own `MBI_calc.m`/`MBI_example.m`). Use either the monolithic `../MBI_calc.m` or this folder, not both on the path.

---

7. Beam-breakup (BBU, test)

Domain dipole-mode BBU .

FILE	SIGNATURE	PURPOSE
`BBU_dipole_tracking.m`	`results = BBU_dipole_tracking(p)`	Track `Nb` bunches through `Nc` cavities / `Na` passes with dipole HOMs; returns growth `decrement`, voltage history, stability. All params optional via struct `p` (defaults in the header). Reads `machines/cavities` & `machines/matrices` files.
`GenerateCavities.m`	`GenerateCavities(hom_file, Nc, varargin)`	Write the per-cavity HOM table (`machines/cavities/XXXX.txt`) BBU tracking needs, from a nominal HOM file; optional frequency/Q spread.
`GenerateMatrices.m`	`GenerateMatrices(beamline, E0_eV, varargin)`	Compute inter-cavity 6×6 transport matrices + arc lengths from a beamline and write `machines/matrices/XXXX.txt`. Same quad-family binding as `TSCenv`.

FILE	SIGNATURE	PURPOSE
`example_BBU_dipole.m`	script	End-to-end synthetic 2-cavity / 4-pass ERL example feeding `BBU_dipole_tracking` (builds `T{}` matrices via a local `fodo_matrix`).

---

8. Other commands

Everything else in `code/` that doesn't belong to a focused section above — utilities, standalone study scripts, and converters. (Intentionally **not** listed: the multi-objective optimization objectives and the space-charge envelope variants other than `TSCenv`.)

`plotFloorPlan.m` — machine floor plan + orbit

```
result = plotFloorPlan(beamline, doPlot, options)
```

Draws the lattice in physical `(x, y)` space: straight elements as rectangles and dipoles as curved segments that **bend the reference axis**, color-coded per element type. Drifts, edges and markers occupy `s` but aren't drawn. Optionally overlays a first-order reference **orbit** and can zoom on it.

- `doPlot` — `1` draws the figure *and* returns the full geometry; any other value returns
  geometry only (no plot).
- `options` (struct, all optional) — appearance: `figure_size` `[600 400]`, `line_width`,
  `axis_line_width`, `font_name` `'Cambria Math'`, `font_size`, `title_text`, `rect_length` (element thickness, def 1.7 m), `rect_width_factor`, `focus_elements`
  (types rendered as shapes, def `{quadrupole,dipole,sextupole,cavity,TGU}`), `ang` (global
  start-rotation, deg). Orbit: `plot_orbit` (false), `initial_orbit` `[x;px;y;py;z;dp]`,
  `initial_gamma` (100), `orbit_color`, `orbit_scale` (transverse magnification, def 50).

- Returns `result.element_positions` (per-element `pos_start`/`pos_center`/`pos_end` `(x,y)`)
  and `result.magnet_centers` (name, type, cumulative `s_center`, `(x,y)`, and center-to-center
  spacing between consecutive magnets).

```
beamline = readBmad('CSRwake/example/pap4.arc');
plotFloorPlan(beamline, 1, struct('rect_length',0.2));   % quick plot
res = plotFloorPlan(beamline, 0);                         % geometry only, no figure
```

Transport & optics utilities

FILE	SIGNATURE	PURPOSE
`getElementMatrix.m`	`[R, gamma_out] = getElementMatrix(el, gamma)`	6×6 first-order matrix for **one** element (the kernel `calculateTotalMatrix` loops over). Cavities use the Rosenzweig–Serafini model and need `gamma`.
`rev_beamline.m`	`rb = rev_beamline(beamline)`	Reverse element order; flips cavity phase by 180°.
`findRijmax.m`	`[minR,maxR,s_min,s_max] = findRijmax(bl, i, j)`	Scan the line (~1000 steps) for the min/max of transfer-matrix element `R(i,j)` and where it occurs.
`trackParticleHighOrder.m`	`p_out = trackParticleHighOrder(p_in, bl, order)`	Track one particle `[x;px;y;py;z;delta]` through **drifts and quads only**, optional 2nd-order terms (`order=2` default).

FILE	SIGNATURE	PURPOSE
`calcsliene.m`	`[val, evolution, bend_avgs] = calcsliene(bl, E_MeV, init_beta, epsx, h0, sigmaz, sigmae, plot_flag)`	Dispersion-function evolution and a slice-emittance / CSR-dispersion figure of merit, averaged over bends.